Name: M . Sivateja

Hall Ticket: 2303A52145

Batch: 41

Task 1: Email Valida on using TDD

Prompt:

Generate comprehensive unit test cases (valid and invalid) for email validation using TDD based on standard email rules. Then implement a Python function is_valid_email(email) that passes all the generated test cases.

Code:

```
[3]
✓ 0s

def is_valid_email(email):
    # This regex aims to cover common valid email formats and exclude many common invalid ones.
    # It balances strictness with practical use, acknowledging that full RFC compliance
    # is extremely complex and often not necessary for typical application validation.

    # Breakdown of the regex:
    # Username Part:
    # ^[a-zA-Z0-9_]+                        : Starts with one or more alphanumeric characters or an underscore.
    #                                         This allows '_email@example.com' (from valid_emails) but prevents
    #                                         '.user@example.com' (from invalid_emails).
    # (?:[._%+-][a-zA-Z0-9_+-]+)*          : Allows '.', '_', '%', '+', '-' as separators between parts of the username.
    #                                         Each separator must be followed by one or more alphanumeric characters,
    #                                         underscores, plus, or hyphens. This prevents consecutive separators
    #                                         (e.g., 'user..name') and ensures the username doesn't end with a separator
    #                                         (e.g., 'user.@example.com').
    #
    # @                                     : The mandatory '@' symbol.
    #
    # Domain Part:
    # ([a-zA-Z0-9-]+\.)+                   : Consists of one or more domain labels, each followed by a dot.
    #                                         Each label must start and end with an alphanumeric character or hyphen
    #                                         (e.g., 'example-domain'). This prevents labels from starting or ending with
    #                                         a hyphen or dot, and disallows consecutive dots (e.g., 'example..com').
    #
    # Top-Level Domain (TLD):
    # [a-zA-Z]{2,63}$                      : The TLD must be composed of 2 to 63 letters and appear at the end of the string.
    #                                         This covers various TLD lengths (e.g., 'com', 'co.uk', 'travel').

    email_regex = re.compile(
        r"^[a-zA-Z0-9_]+(?:[._%+-][a-zA-Z0-9_+-]+)*"
        r"@"
        r"([a-zA-Z0-9-]+\.)+"
        r"[a-zA-Z]{2,63}$"
    )

    return bool(email_regex.match(email))

print("The `is_valid_email` function has been defined.")
```

[3]
0s

```python
print("The `is_valid_email` function has been defined.")
```

```
The `is_valid_email` function has been defined.
```

Reasoning: Now that the `is_valid_email` function is defined, I will test its implementation using the previously generated `valid_emails` and `invalid_emails` lists to verify its correctness.

[4]
0s

```python
print("\n--- Testing Valid Emails ---")
correct_valid_count = 0
for email in valid_emails:
    is_valid = is_valid_email(email)
    if is_valid:
        print(f"'{email}': VALID (Correct)")
        correct_valid_count += 1
    else:
        print(f"'{email}': INVALID (Incorrect)")

print(f"\nCorrectly identified valid emails: {correct_valid_count}/{len(valid_emails)}")

print("\n--- Testing Invalid Emails ---")
correct_invalid_count = 0
for email in invalid_emails:
    is_valid = is_valid_email(email)
    if not is_valid:
        print(f"'{email}': INVALID (Correct)")
        correct_invalid_count += 1
    else:
        print(f"'{email}': VALID (Incorrect)")

print(f"\nCorrectly identified invalid emails: {correct_invalid_count}/{len(invalid_emails)}")

# Overall summary
total_emails = len(valid_emails) + len(invalid_emails)
total_correct = correct_valid_count + correct_invalid_count
print(f"\nOverall Accuracy: {total_correct}/{total_emails} ({total_correct/total_emails:.2%})")
```

Output:

```
--- Testing Valid Emails ---
'test@example.com': VALID (Correct)
'firstname.lastname@example.com': VALID (Correct)
'email@subdomain.example.com': VALID (Correct)
'firstname+tag@example.com': VALID (Correct)
'1234567890@example.com': VALID (Correct)
'email@example-domain.com': VALID (Correct)
'email@domain.co.uk': VALID (Correct)
'email@example.org': VALID (Correct)
'email@example.net': VALID (Correct)
'user@domain.info': VALID (Correct)
'a@b.cd': VALID (Correct)
'long.username.with.many.parts@example.com': VALID (Correct)
'email@123.com': VALID (Correct)
'_email@example.com': VALID (Correct)
'email@example.travel': VALID (Correct)
'email@example.museum': VALID (Correct)
'email@example.name': VALID (Correct)
'email@example.pro': VALID (Correct)
'email@example.aero': VALID (Correct)
'email@example.coop': VALID (Correct)
'email@example.mobi': VALID (Correct)

Correctly identified valid emails: 21/21

--- Testing Invalid Emails ---
'test.example.com': INVALID (Correct)
'user@': INVALID (Correct)
'@example.com': INVALID (Correct)
'user name@example.com': INVALID (Correct)
'user@example com': INVALID (Correct)
'user@.com': INVALID (Correct)
'user@example..com': INVALID (Correct)
'user@example': INVALID (Correct)
'user@example_domain.com': INVALID (Correct)
'user@example,com': INVALID (Correct)
'.user@example.com': INVALID (Correct)
'user.@example.com': INVALID (Correct)
'user@example.c': INVALID (Correct)
'user@example.c.om': VALID (Incorrect)
'user@@example.com': INVALID (Correct)
```

```
'user name@example.com': INVALID (Correct)
'user@example com': INVALID (Correct)
'user@.com': INVALID (Correct)
'user@example..com': INVALID (Correct)
'user@example': INVALID (Correct)
'user@example_domain.com': INVALID (Correct)
'user@example,com': INVALID (Correct)
'.user@example.com': INVALID (Correct)
'user.@example.com': INVALID (Correct)
'user@example.c': INVALID (Correct)
'user@example.c.om': VALID (Incorrect)
'user@@example.com': INVALID (Correct)
'user@%example.com': INVALID (Correct)
'#user@example.com': INVALID (Correct)
'': INVALID (Correct)
'plainstring': INVALID (Correct)
'a@b': INVALID (Correct)
'user@example.c0m': INVALID (Correct)
'user@local': INVALID (Correct)
'invalid-email': INVALID (Correct)
'justtext': INVALID (Correct)
'not_an_email.com': INVALID (Correct)
'user@domain_with_underscore.com': INVALID (Correct)
'user@domain.com.': INVALID (Correct)
'user@.domain.com': INVALID (Correct)
'user@domain..com': INVALID (Correct)

Correctly identified invalid emails: 28/29

Overall Accuracy: 49/50 (98.00%)
```

Final Task

Subtask:

Summarize the test results, indicating how many tests passed and failed, and confirm the functionality of the `is_valid_email` function.

Explanatoin:

The code checks if an email is valid by making sure it contains exactly one “@”, includes required symbols like a dot in the domain, and places special characters correctly. It was developed using Test-Driven Development (TDD), meaning the tests were written first and the code was built to pass them.

Task 2: Grade Assignment using Loops

Python Code for Bubble Sort:

Prompt:

Generate unit test cases using TDD for assign_grade(score) where 90–100 → A, 80–89 → B, 70–79 → C, 60–69 → D, below 60 → F, including boundary and invalid inputs. Then implement the Python function assign_grade(score) to pass all generated test cases.

Code:

```python
valid_test_cases = [
    # Grade A (90-100)
    {'score': 100, 'expected_grade': 'A'},
    {'score': 95, 'expected_grade': 'A'},
    {'score': 90, 'expected_grade': 'A'},

    # Grade B (80-89)
    {'score': 89, 'expected_grade': 'B'},
    {'score': 85, 'expected_grade': 'B'},
    {'score': 80, 'expected_grade': 'B'},

    # Grade C (70-79)
    {'score': 79, 'expected_grade': 'C'},
    {'score': 75, 'expected_grade': 'C'},
    {'score': 70, 'expected_grade': 'C'},

    # Grade D (60-69)
    {'score': 69, 'expected_grade': 'D'},
    {'score': 65, 'expected_grade': 'D'},
    {'score': 60, 'expected_grade': 'D'},

    # Grade F (0-59)
    {'score': 59, 'expected_grade': 'F'},
    {'score': 50, 'expected_grade': 'F'},
    {'score': 0, 'expected_grade': 'F'}
]

invalid_test_cases = [
    # Scores below 0
    {'score': -1, 'expected_error': 'Invalid score: Score must be between 0 and 100'},
    {'score': -50, 'expected_error': 'Invalid score: Score must be between 0 and 100'},
    {'score': -0.1, 'expected_error': 'Invalid score: Score must be between 0 and 100'},

    # Scores above 100
    {'score': 101, 'expected_error': 'Invalid score: Score must be between 0 and 100'},
    {'score': 150, 'expected_error': 'Invalid score: Score must be between 0 and 100'},
    {'score': 100.1, 'expected_error': 'Invalid score: Score must be between 0 and 100'},
```

```python
    # Non-numeric types
    {'score': 'abc', 'expected_error': 'Invalid score: Score must be a number'},
    {'score': 'ninety', 'expected_error': 'Invalid score: Score must be a number'},
    {'score': None, 'expected_error': 'Invalid score: Score must be a number'},
    {'score': True, 'expected_error': 'Invalid score: Score must be a number'},
    {'score': False, 'expected_error': 'Invalid score: Score must be a number'},
    {'score': [], 'expected_error': 'Invalid score: Score must be a number'},
    {'score': {}, 'expected_error': 'Invalid score: Score must be a number'}
]

def assign_grade(score):
    # Check if the input score is a numeric type, explicitly excluding boolean
    if not isinstance(score, (int, float)) or isinstance(score, bool):
        return 'Invalid score: Score must be a number'

    # Check if the score is within the valid range of 0 to 100
    if not (0 <= score <= 100):
        return 'Invalid score: Score must be between 0 and 100'

    # Implement the grading logic
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'
```

```python
valid_tests_passed = 0
valid_tests_failed = 0
invalid_tests_passed = 0
invalid_tests_failed = 0

print("\n--- Running Valid Test Cases ---")
for i, case in enumerate(valid_test_cases):
    score = case['score']
    expected_grade = case['expected_grade']
    actual_grade = assign_grade(score)

    if actual_grade == expected_grade:
        print(f"Valid Test Case {i+1}: Score {score}, Expected '{expected_grade}', Got '{actual_grade}' - PASSED")
        valid_tests_passed += 1
    else:
        print(f"Valid Test Case {i+1}: Score {score}, Expected '{expected_grade}', Got '{actual_grade}' - FAILED")
        valid_tests_failed += 1

print("\n--- Running Invalid Test Cases ---")
for i, case in enumerate(invalid_test_cases):
    score = case['score']
    expected_error = case['expected_error']
    actual_result = assign_grade(score)

    if actual_result == expected_error:
        print(f"Invalid Test Case {i+1}: Score {score}, Expected '{expected_error}', Got '{actual_result}' - PASSED")
        invalid_tests_passed += 1
    else:
        print(f"Invalid Test Case {i+1}: Score {score}, Expected '{expected_error}', Got '{actual_result}' - FAILED")
        invalid_tests_failed += 1

print("\n--- Test Summary ---")
print(f"Total Valid Tests: {len(valid_test_cases)}")
print(f"Valid Tests Passed: {valid_tests_passed}")
print(f"Valid Tests Failed: {valid_tests_failed}")
print(f"Total Invalid Tests: {len(invalid_test_cases)}")
print(f"Invalid Tests Passed: {invalid_tests_passed}")
print(f"Invalid Tests Failed: {invalid_tests_failed}")
```

```python
print(f"\nOverall Tests Run: {total_tests}")
print(f"Overall Tests Passed: {total_passed}")
print(f"Overall Tests Failed: {total_failed}")
```

Output:

```
--- Running Valid Test Cases ---
Valid Test Case 1: Score 100, Expected 'A', Got 'A' - PASSED
Valid Test Case 2: Score 95, Expected 'A', Got 'A' - PASSED
Valid Test Case 3: Score 90, Expected 'A', Got 'A' - PASSED
Valid Test Case 4: Score 89, Expected 'B', Got 'B' - PASSED
Valid Test Case 5: Score 85, Expected 'B', Got 'B' - PASSED
Valid Test Case 6: Score 80, Expected 'B', Got 'B' - PASSED
Valid Test Case 7: Score 79, Expected 'C', Got 'C' - PASSED
Valid Test Case 8: Score 75, Expected 'C', Got 'C' - PASSED
Valid Test Case 9: Score 70, Expected 'C', Got 'C' - PASSED
Valid Test Case 10: Score 69, Expected 'D', Got 'D' - PASSED
Valid Test Case 11: Score 65, Expected 'D', Got 'D' - PASSED
Valid Test Case 12: Score 60, Expected 'D', Got 'D' - PASSED
Valid Test Case 13: Score 59, Expected 'F', Got 'F' - PASSED
Valid Test Case 14: Score 50, Expected 'F', Got 'F' - PASSED
Valid Test Case 15: Score 0, Expected 'F', Got 'F' - PASSED

--- Running Invalid Test Cases ---
Invalid Test Case 1: Score -1, Expected 'Invalid score: Score must be between 0 and 100', Got 'Invalid score: Score must be between 0 and 100' - PASSED
Invalid Test Case 2: Score -50, Expected 'Invalid score: Score must be between 0 and 100', Got 'Invalid score: Score must be between 0 and 100' - PASSED
Invalid Test Case 3: Score -0.1, Expected 'Invalid score: Score must be between 0 and 100', Got 'Invalid score: Score must be between 0 and 100' - PASSED
Invalid Test Case 4: Score 101, Expected 'Invalid score: Score must be between 0 and 100', Got 'Invalid score: Score must be between 0 and 100' - PASSED
Invalid Test Case 5: Score 150, Expected 'Invalid score: Score must be between 0 and 100', Got 'Invalid score: Score must be between 0 and 100' - PASSED
Invalid Test Case 6: Score 100.1, Expected 'Invalid score: Score must be between 0 and 100', Got 'Invalid score: Score must be between 0 and 100' - PASSED
Invalid Test Case 7: Score abc, Expected 'Invalid score: Score must be a number', Got 'Invalid score: Score must be a number' - PASSED
Invalid Test Case 8: Score ninety, Expected 'Invalid score: Score must be a number', Got 'Invalid score: Score must be a number' - PASSED
Invalid Test Case 9: Score None, Expected 'Invalid score: Score must be a number', Got 'Invalid score: Score must be a number' - PASSED
Invalid Test Case 10: Score True, Expected 'Invalid score: Score must be a number', Got 'Invalid score: Score must be a number' - PASSED
Invalid Test Case 11: Score False, Expected 'Invalid score: Score must be a number', Got 'Invalid score: Score must be a number' - PASSED
Invalid Test Case 12: Score [], Expected 'Invalid score: Score must be a number', Got 'Invalid score: Score must be a number' - PASSED
Invalid Test Case 13: Score {}, Expected 'Invalid score: Score must be a number', Got 'Invalid score: Score must be a number' - PASSED

--- Test Summary ---
Total Valid Tests: 15
Valid Tests Passed: 15
Valid Tests Failed: 0
Total Invalid Tests: 13
Invalid Tests Passed: 13
Invalid Tests Failed: 0
```

Explanation:

The function determines letter grades by comparing a score to defined ranges, making sure boundary values (like exact cutoffs) are handled correctly. It also checks for invalid inputs and handles them safely to ensure accurate and reliable grading.

Task 3: Sentence Palindrome Checker

Prompt:

Generate comprehensive unit test cases for is_sentence_palindrome(sentence) that ignore case, spaces, and punctua on, covering both palindromic and non-palindromic sentences. Then implement the Python function is_sentence_palindrome(sentence) so it passes all the generated test cases.

Code:

and edge cases.

```python
import re

palindromic_sentences = [
    "Madam, in Eden, I'm Adam.",
    "A man, a plan, a canal: Panama.",
    "No lemon, no melon.",
    "Race fast, safe car.",
    "Was it a car or a cat I saw?",
    "Eva, can I see bees in a cave?",
    "Live on no evil.",
    "No 'x' in Nixon.",
    "A Toyota's a Toyota.",
    "Mr. Owl ate my metal worm.",
    "Don't nod.",
    "I did, did I?",
    "My gym.",
    "Able was I ere I saw Elba.",
    "Rats live on no evil star.",
    "Step on no pets."
]

non_palindromic_sentences = [
    "This is not a palindrome.",
    "Hello, world!",
    "Python programming is fun.",
    "A quick brown fox jumps over the lazy dog.",
    "The quick brown fox.",
    "Never odd or even, but this sentence is not.",
    "I enjoy coding in Jupyter notebooks.",
    "Data science is an interesting field.",
    "Palindrome test case generation is complete.",
    "A B C D E F G H I J K L M N O P Q R S T U V W X Y Z",
    "Random words create non palindromes.",
    "This sentence contains multiple spaces and punctuation, but is not a palindrome.",
    "Another example of a non-palindrome.",
    "The cat sat on the mat.",
    "Testing for palindromes is crucial."
```

```python
]

def is_sentence_palindrome(sentence):
    """
    Checks if a sentence is a palindrome, ignoring case, spaces, and punctuation.

    Args:
        sentence (str): The input sentence.

    Returns:
        bool: True if the sentence is a palindrome, False otherwise.
    """
    # Remove non-alphanumeric characters and convert to lowercase
    cleaned_sentence = re.sub(r'[^a-zA-Z0-9]', '', sentence).lower()

    # Check if the cleaned string is equal to its reverse
    return cleaned_sentence == cleaned_sentence[::-1]

print("Running unit tests...")

palindromic_passed = 0
palindromic_failed = 0

for sentence in palindromic_sentences:
    result = is_sentence_palindrome(sentence)
    if result:
        palindromic_passed += 1
        print(f"PALINDROME TEST - PASSED: '{sentence}' is correctly identified as a palindrome.")
    else:
        palindromic_failed += 1
        print(f"PALINDROME TEST - FAILED: '{sentence}' is incorrectly identified as NOT a palindrome.")
```

```python
non_palindromic_passed = 0
non_palindromic_failed = 0

for sentence in non_palindromic_sentences:
    result = is_sentence_palindrome(sentence)
    if not result:
        non_palindromic_passed += 1
        print(f"NON-PALINDROME TEST - PASSED: '{sentence}' is correctly identified as NOT a palindrome.")
    else:
        non_palindromic_failed += 1
        print(f"NON-PALINDROME TEST - FAILED: '{sentence}' is incorrectly identified as a palindrome.")

print("\n--- Test Summary ---")
print(f"Palindromic Sentences: {len(palindromic_sentences)} total")
print(f"  Passed: {palindromic_passed}")
print(f"  Failed: {palindromic_failed}")

print(f"Non-Palindromic Sentences: {len(non_palindromic_sentences)} total")
print(f"  Passed: {non_palindromic_passed}")
print(f"  Failed: {non_palindromic_failed}")

total_passed = palindromic_passed + non_palindromic_passed
total_failed = palindromic_failed + non_palindromic_failed
total_tests = len(palindromic_sentences) + len(non_palindromic_sentences)

print(f"\nOverall Test Results: {total_passed}/{total_tests} tests passed ({total_failed} failed).")

if total_failed == 0:
    print("All tests passed successfully!")
else:
    print("Some tests failed. Review the output above for details.")
```

Output:

```
Running unit tests...
PALINDROME TEST - PASSED: 'Madam, in Eden, I'm Adam.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'A man, a plan, a canal: Panama.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'No lemon, no melon.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'Race fast, safe car.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'Was it a car or a cat I saw?' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'Eva, can I see bees in a cave?' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'Live on no evil.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'No 'x' in Nixon.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'A Toyota's a Toyota.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'Mr. Owl ate my metal worm.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'Don't nod.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'I did, did I?' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'My gym.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'Able was I ere I saw Elba.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'Rats live on no evil star.' is correctly identified as a palindrome.
PALINDROME TEST - PASSED: 'Step on no pets.' is correctly identified as a palindrome.
NON-PALINDROME TEST - PASSED: 'This is not a palindrome.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'Hello, world!' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'Python programming is fun.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'A quick brown fox jumps over the lazy dog.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'The quick brown fox.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'Never odd or even, but this sentence is not.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'I enjoy coding in Jupyter notebooks.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'Data science is an interesting field.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'Palindrome test case generation is complete.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'A B C D E F G H I J K L M N O P Q R S T U V W X Y Z' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'Random words create non palindromes.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'This sentence contains multiple spaces and punctuation, but is not a palindrome.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'Another example of a non-palindrome.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'The cat sat on the mat.' is correctly identified as NOT a palindrome.
NON-PALINDROME TEST - PASSED: 'Testing for palindromes is crucial.' is correctly identified as NOT a palindrome.

--- Test Summary ---
Palindromic Sentences: 16 total
  Passed: 16
  Failed: 0
Non-Palindromic Sentences: 15 total
  Passed: 15
  Failed: 0

Overall Test Results: 31/31 tests passed (0 failed).
All tests passed successfully!
```

Explanation:

The module manages cart items by allowing products to be added or removed while keeping the cart updated correctly. It also calculates the total cost accurately and safely handles cases like removing non-existing items or when the cart is empty.

Task 5: Date Format Conversion

Prompt:

Generate TDD-based unit test cases for convert_date_format(date_str) that converts dates from "YYYY-MM-DD" to "DD-MM-YYYY", then implement the Python function to pass all generated tests.

Code:

```python
import re
import sys

# 1. Define valid_date_test_cases
valid_date_test_cases = [
    # Typical dates
    {'input': '2023-01-15', 'expected_output': '15-01-2023'},
    {'input': '2022-07-20', 'expected_output': '20-07-2022'},

    # Dates at the beginning and end of months
    {'input': '2023-03-01', 'expected_output': '01-03-2023'},
    {'input': '2023-03-31', 'expected_output': '31-03-2023'},
    {'input': '2023-04-01', 'expected_output': '01-04-2023'},
    {'input': '2023-04-30', 'expected_output': '30-04-2023'},

    # Dates with single-digit months and days
    {'input': '2023-01-01', 'expected_output': '01-01-2023'},
    {'input': '2023-09-05', 'expected_output': '05-09-2023'},
    {'input': '2023-05-09', 'expected_output': '09-05-2023'},

    # Leap years
    {'input': '2024-02-29', 'expected_output': '29-02-2024'}, # Leap year
    {'input': '2000-02-29', 'expected_output': '29-02-2000'}, # Century leap year

    # Non-leap years
    {'input': '2023-02-28', 'expected_output': '28-02-2023'},
    {'input': '1900-02-28', 'expected_output': '28-02-1900'}, # Century non-leap year

    # Dates spanning different centuries/millennia
    {'input': '1900-12-31', 'expected_output': '31-12-1900'},
    {'input': '2000-01-01', 'expected_output': '01-01-2000'},
    {'input': '1999-12-31', 'expected_output': '31-12-1999'},
    {'input': '2100-01-01', 'expected_output': '01-01-2100'}
]
```

```python
# 2. Define invalid_date_test_cases
invalid_date_test_cases = [
    # Incorrect date formats
    {'input': '2023/01/15', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '15-01-2023', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '01/15/2023', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '20230115', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '2023-1-1', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '2023-01-5', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '2023-Jan-15', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '2023.01.15', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},

    # Non-existent dates
    {'input': '2023-02-30', 'expected_error': 'Non-existent date.'},
    {'input': '2023-13-01', 'expected_error': 'Non-existent date.'},
    {'input': '2023-04-31', 'expected_error': 'Non-existent date.'},
    {'input': '2023-06-31', 'expected_error': 'Non-existent date.'},
    {'input': '2023-09-31', 'expected_error': 'Non-existent date.'},
    {'input': '2023-11-31', 'expected_error': 'Non-existent date.'},
    {'input': '2023-00-01', 'expected_error': 'Non-existent date.'},
    {'input': '2023-01-00', 'expected_error': 'Non-existent date.'},
    {'input': '2023-02-29', 'expected_error': 'Non-existent date.'}, # Non-leap year
    {'input': '1900-02-29', 'expected_error': 'Non-existent date.'}, # Non-leap century year

    # Non-string inputs
    {'input': None, 'expected_error': 'Input must be a string.'},
    {'input': 12345, 'expected_error': 'Input must be a string.'},
    {'input': [2023, 1, 1], 'expected_error': 'Input must be a string.'},
    {'input': {'date': '2023-01-01'}, 'expected_error': 'Input must be a string.'},
    {'input': True, 'expected_error': 'Input must be a string.'},

    # Empty string and whitespace-only string
    {'input': '', 'expected_error': 'Date string cannot be empty.'},
    {'input': '   ', 'expected_error': 'Date string cannot be empty.'},
```

```python
    # Dates with too many or too few digits/parts
    {'input': '2023-01-01-01', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '23-01-01', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '2023-1-15', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '2023-01-1', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '2023-01', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '2023', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '01-01', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '01', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'},
    {'input': '2023-1-1-1', 'expected_error': 'Invalid date format. Expected YYYY-MM-DD.'}
]

# 3. Implement convert_date_format function
def convert_date_format(date_str):
    # 2. Check if date_str is a string
    if not isinstance(date_str, str):
        return 'Input must be a string.'

    # 3. Check if date_str is empty or contains only whitespace
    if not date_str.strip():
        return 'Date string cannot be empty.'

    # 4. Use a regular expression to validate 'YYYY-MM-DD' format
    match = re.match(r'^(\d{4})-(\d{2})-(\d{2})$', date_str)
    if not match:
        return 'Invalid date format. Expected YYYY-MM-DD.'

    # 5. Extract year, month, and day components as integers
    year, month, day = map(int, match.groups())

    # 6. Implement date validity checks
    # a. Ensure the month is between 1 and 12.
    if not (1 <= month <= 12):
        return 'Non-existent date.'

    # Days in each month (non-leap year)
    days_in_month = [
        0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
    ]
```

```python
    # Check for leap year
    is_leap_year = False
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        is_leap_year = True
        days_in_month[2] = 29 # February has 29 days in a leap year

    # b. Ensure the day is valid for the given month and year
    if not (1 <= day <= days_in_month[month]):
        return 'Non-existent date.'

    # 8. Reformat the date components into 'DD-MM-YYYY' string format
    return f"{day:02d}-{month:02d}-{year:04d}"

 4. Execute Tests and Summarize Results
rint("\n--- Running Valid Date Tests ---")
alid_passed_count = 0
alid_failed_count = 0
or i, case in enumerate(valid_date_test_cases):
    input_str = case['input']
    expected_output = case['expected_output']
    actual_output = convert_date_format(input_str)

    if actual_output == expected_output:
        valid_passed_count += 1
    else:
        valid_failed_count += 1
        print(f"FAIL (Valid Case {i+1}): Input: '{input_str}', Expected: '{expected_output}', Got: '{actual_output}'")

rint("\n--- Running Invalid Date Tests ---")
nvalid_passed_count = 0
nvalid_failed_count = 0
or i, case in enumerate(invalid_date_test_cases):
    input_str = case['input']
    expected_error = case['expected_error']
    actual_error = convert_date_format(input_str)

    if actual_error == expected_error:
        invalid_passed_count += 1
    else:
        invalid_failed_count += 1
        print(f"FAIL (Invalid Case {i+1}): Input: '{input_str}', Expected Error: '{expected_error}', Got: '{actual_error}'")
```

```python
# --- Summarize Results ---
total_valid_tests = len(valid_date_test_cases)
total_invalid_tests = len(invalid_date_test_cases)
total_tests = total_valid_tests + total_invalid_tests

print("\n--- Test Summary ---")
print(f"Valid Date Tests:")
print(f"  Passed: {valid_passed_count}/{total_valid_tests}")
print(f"  Failed: {valid_failed_count}/{total_valid_tests}")

print(f"\nInvalid Date Tests:")
print(f"  Passed: {invalid_passed_count}/{total_invalid_tests}")
print(f"  Failed: {invalid_failed_count}/{total_invalid_tests}")

total_passed = valid_passed_count + invalid_passed_count
total_failed = valid_failed_count + invalid_failed_count

print(f"\nOverall Test Results:")
print(f"  Total Passed: {total_passed}/{total_tests}")
print(f"  Total Failed: {total_failed}/{total_tests}")

if total_failed == 0:
    print("  All tests passed successfully!")
else:
    print("  Some tests failed. Please review the failed cases above.")

# Exit with a non-zero code if any tests failed for CI/CD like environments
if total_failed > 0:
    sys.exit(1)
```

Output:

```
--- Running Valid Date Tests ---

--- Running Invalid Date Tests ---

--- Test Summary ---
Valid Date Tests:
  Passed: 17/17
  Failed: 0/17

Invalid Date Tests:
  Passed: 34/34
  Failed: 0/34

Overall Test Results:
  Total Passed: 51/51
  Total Failed: 0/51
  All tests passed successfully!
```

Reasoning: I need to generate a list of valid date test cases according to the instructions, covering various scenarios like typical dates, boundary cases, leap years, and dates across centuries. I will store these in a list of dictionaries.

Explanation:
The function takes a valid date in year–month–day format and rearranges it into day–month–year format. It ensures the original date values remain correct while only changing their order.